

# HHBKTendo – Spielesammlung

*HHBK Düsseldorf · Lernfeld 5 · 2026*

---

- Strategiespiele mit MiniMax-KI
- Bauernschach & Tic-Tac-Toe (4 gewinnt)
- Tech-Stack: Python 3.10+ · SQLite · tkinter

# HHBKTendo – Spielesammlung

## *Eröffnung*

---

Guten Tag, wir präsentieren heute **HHBKTendo** – eine Spielesammlung, die im Rahmen von Lernfeld 5 an der HHBK Düsseldorf entwickelt wurde.

Das Projekt besteht aus zwei Strategiespielen – **Bauernschach** und **Tic-Tac-Toe (4 gewinnt)** – jeweils auf einem 6×6-Spielfeld, gegen eine KI die auf dem MiniMax-Algorithmus basiert.

Technisch setzt das Projekt auf **Python 3.10+**, eine **SQLite-Datenbank** und eine **tkinter-Oberfläche** – alles ohne externe Abhängigkeiten außer Pillow für Animationen.

■ **Kurze Vorstellung des Teams vor dem Start, dann direkt in den Überblick.**

## Projektüberblick

---

### ■ Spielesammlung

- 2 Spiele auf je 6×6-Feld

### ■ KI-Gegner

- MiniMax + Alpha-Beta-Pruning
- Kein Code-Duplikat für beide Spiele

### ■ Benutzerverwaltung

- Login / Registrierung · SHA-256-Hashing · Gast-Modus

### ■ Bestenliste

- SQLite · Rangliste nach Schwierigkeitsgrad

## Projektüberblick

---

Das Projekt hat vier Kernbereiche:

- **Spielesammlung:** Zwei vollständige Strategiespiele auf einem 6×6-Feld, komplett spielbar gegen eine KI.
- **KI-Gegner:** Ein einziger, wiederverwendbarer MiniMax-Algorithmus mit Alpha-Beta-Pruning für beide Spiele – kein duplizierter Code.
- **Benutzerverwaltung:** Registrierung und Login mit SHA-256-Hashing und Salt. Alternativ: Gast-Modus ohne Konto.
- **Bestenliste:** Alle Spielergebnisse werden in SQLite gespeichert und können nach Spiel und Schwierigkeitsgrad abgerufen werden.
- **Überblick-Folie – ruhig etwas langsamer sprechen, damit alle folgen können.**

## Projektstruktur

---

- main.py → GUI, Screens, app\_state
- minimax.py → generischer MiniMax + Alpha-Beta
- pawn\_chess.py → Bauernschach Züge & Bewertung
- tictactoe.py → Tic-Tac-Toe Züge & Bewertung
- auth.py → Login & Registrierung (SHA-256 + Salt)
- database.py → SQLite CRUD (User & Ergebnisse)
- test\_all.py → 15 Unittests (TC-01 bis TC-15)
- games.db → wird automatisch erstellt

## Projektstruktur

### *Modularer Aufbau*

---

Das Projekt ist in sieben Python-Dateien aufgeteilt, jede mit einer klar definierten Aufgabe:

- **main.py** – Der GUI-Einstiegspunkt. Verwaltet alle Screens und den globalen Zustand über das `app_state`-Dictionary.
  - **minimax.py** – Die KI-Engine. Generischer MiniMax mit Alpha-Beta-Pruning, nutzbar für jedes Spiel per Callbacks.
  - **pawn\_chess.py / tictactoe.py** – Die eigentliche Spiellogik: erlaubte Züge, Anwenden, Gewinner, Bewertung.
  - **auth.py** – Registrierung und Login mit SHA-256-Hashing und Salt.
  - **database.py** – Alle Datenbankoperationen: Nutzer anlegen, Ergebnisse speichern, Bestenliste abfragen.
  - **test\_all.py** – 15 Unittests (TC-01 bis TC-15), pytest-kompatibel.
- **Auf die klare Trennung der Zuständigkeiten hinweisen – jede Datei hat genau eine Aufgabe.**

# Benutzerverwaltung & Datenbank

*auth.py · database.py*

---

## auth.py – Kernfunktionen

- register() → Validierung + SHA-256 + Salt
- login() → Passwort-Hash-Vergleich
- save/load\_session() → session.json für Auto-Login
- logout() → Session löschen
- is\_guest() → Gast = keine Statistiken

## SQLite-Tabellen

- users: id, username, password\_hash, language, created\_at
- results: id, user\_id (FK), game, difficulty, won, played\_at

# Benutzerverwaltung & Datenbank

*auth.py · database.py*

---

## auth.py – Funktionen

- **register()**: Prüft ob der Username schon vergeben ist, hasht das Passwort mit SHA-256 + Salt und speichert den neuen Nutzer in der DB.
- **login()**: Lädt den gespeicherten Hash aus der DB, wendet denselben Salt an und vergleicht das Ergebnis.
- **save\_session() / load\_session()**: Schreiben bzw. lesen die user\_id in eine session.json – so bleibt man beim nächsten Start eingeloggt.
- **logout()**: Löscht die session.json und setzt den app\_state zurück.
- **is\_guest()**: Gibt True zurück wenn kein Nutzer eingeloggt ist – Gäste sehen keine Statistiken.

## SQLite-Tabellen

- **users**: id, username, password\_hash, language, created\_at
  - **results**: id, user\_id (Fremdschlüssel), game, difficulty, won, played\_at
- Kurz erwähnen dass die DB automatisch erstellt wird wenn sie noch nicht existiert.



## DB-Techniken

*row\_factory · Schema-Migration · SQL-Injection-Schutz*

---

- `sqlite3.Row` → Spaltenzugriff per Name: `row['username']`
- `dict(row)` → einfache Konvertierung für die App
- `?`-Platzhalter → schützt vor SQL-Injection
- `try/except` bei `ALTER TABLE` → DB bleibt kompatibel

## Datenbank-Techniken

*row\_factory · Migration · SQL-Injection-Schutz*

---

Wir nutzen drei wichtige Techniken für eine robuste Datenbankanbindung:

### sqlite3.Row als row\_factory

Statt Spalten per Index anzusprechen (`row[1]`) kann man sie per Name abrufen: `row['username']`. Das macht den Code lesbarer und weniger fehleranfällig. Mit `dict(row)` lässt sich das Ergebnis einfach in ein Dictionary umwandeln.

### SQL-Injection-Schutz

Alle SQL-Abfragen verwenden **?-Platzhalter** statt String-Konkatenation. SQLite ersetzt die Platzhalter sicher und verhindert so, dass Nutzereingaben als SQL-Code interpretiert werden.

### Schema-Migration

ALTER TABLE läuft in einem try/except-Block. Falls die Spalte schon existiert, wird der Fehler ignoriert – so bleibt die DB abwärtskompatibel.

■ **Gutes Beispiel für defensives Programmieren – kurz den Sicherheitsaspekt betonen.**

## Sichere Passwortspeicherung

SHA-256 + Salt

---

- Kein Klartext – nur 'salt:hash' in der DB
- `os.urandom(16)` → 16 kryptografisch zufällige Bytes
- Salt macht Rainbow-Tables wirkungslos
- Gleiche Passwörter → verschiedene Hashes
- SHA-256 ist Einweg-Funktion

## Sichere Passwortspeicherung

### SHA-256 + Salt

---

Passwörter werden niemals im Klartext gespeichert. Stattdessen wird ein **kryptografischer Hash** mit einem zufälligen Salt erzeugt:

- **os.urandom(16)** erzeugt 16 kryptografisch zufällige Bytes als Salt. Ein zufälliger Salt sorgt dafür, dass zwei gleiche Passwörter unterschiedliche Hashes erzeugen.
- **Rainbow-Tables** sind vorberechnete Tabellen mit Passwort-Hash-Paaren. Durch den einzigartigen Salt wird diese Angriffsmethode wirkungslos.
- **SHA-256** ist eine Einweg-Hashfunktion – aus dem Hash lässt sich das ursprüngliche Passwort nicht zurückberechnen.
- In der Datenbank wird nur **'salt:hash'** gespeichert – niemals das Passwort selbst.

Beim Login wird der gespeicherte Salt verwendet, das eingegebene Passwort erneut gehasht und das Ergebnis verglichen.

■ **Falls jemand fragt: SHA-256 ist für dieses Schulprojekt ausreichend. Produktive Systeme würden bcrypt oder argon2 verwenden.**

# MiniMax-Algorithmus

*mit Alpha-Beta-Pruning*

---

- Rekursive Spielbaumsuche bis zur Tiefe N
- KI = Maximizer · Mensch = Minimizer
- Alpha-Beta: irrelevante Äste werden abgeschnitten

## Schwierigkeitsgrade → Suchtiefe

- 1 Leicht = Tiefe 1 | 2 Mittel = Tiefe 2
- 3 Schwer = Tiefe 3 | 4 Experte = Tiefe 4
- 5 Meister = Tiefe 5 (sehr stark, mehrere Sekunden)

# MiniMax-Algorithmus

*mit Alpha-Beta-Pruning*

Der MiniMax-Algorithmus ist die KI-Grundlage beider Spiele. Er funktioniert wie folgt:

- Der Algorithmus baut einen **Spielbaum** auf – er simuliert alle möglichen Züge bis zu einer festgelegten Tiefe und bewertet jeden Endzustand.
- Es gibt zwei Spieler: den **Maximizer** (die KI, die einen möglichst hohen Score will) und den **Minimizer** (der Mensch, der den Score möglichst niedrig halten will). Beide spielen optimal.
- **Alpha-Beta-Pruning** ist eine Optimierung: Äste des Spielbaums, die das Ergebnis nicht mehr beeinflussen können, werden abgeschnitten. Das spart Rechenzeit deutlich.

## Schwierigkeitsgrade = Suchtiefe

- Tiefe 1 (Leicht) – kaum strategisch | Tiefe 3 (Schwer) – Standard
  - Tiefe 5 (Meister) – sehr stark, kann mehrere Sekunden dauern
- **Visualisierung: Spielbaum mit Max/Min-Ebenen kurz erklären, dann auf Alpha-Beta eingehen.**

## Callback-basiertes MiniMax-Design

---

- `get_best_move()` erhält Callbacks als Parameter:
  - `get_moves_fn` → welche Züge gibt es?
  - `apply_move_fn` → Zug ausführen
  - `evaluate_fn` → Bewertung des Zustands
- Kein Code-Duplikat: ein MiniMax für beide Spiele
- Single Responsibility: KI- & Spiellogik getrennt
- Einfach erweiterbar: drittes Spiel = neue Callbacks

## Callback-basiertes MiniMax-Design

*Eine KI-Engine für alle Spiele*

---

Das Herzstück des Designs: Die MiniMax-Funktion `get_best_move()` kennt kein spezifisches Spiel. Stattdessen bekommt sie drei Callback-Funktionen übergeben:

- **get\_moves\_fn** – gibt die Liste aller gültigen Züge für den aktuellen Spielzustand zurück.
- **apply\_move\_fn** – wendet einen Zug auf das Spielfeld an und gibt den neuen Zustand zurück.
- **evaluate\_fn** – bewertet einen Spielzustand mit einem numerischen Score.

Diese drei Funktionen werden für jedes Spiel separat implementiert. Der MiniMax-Code selbst muss dafür nicht verändert werden.

### Vorteile

- Kein duplizierter Code – dieselbe KI-Logik für beide Spiele.
- Single Responsibility – KI und Spiellogik vollständig getrennt.
- Einfach erweiterbar – ein drittes Spiel braucht nur neue Callbacks.
- **Gutes Beispiel für Clean Code und das Open/Closed Principle.**



## Bauernschach

6×6 · *Weiß (Mensch) vs. Schwarz (KI)*

---

### Spielfeld

- 6×6 Felder, nur Bauern · Weiß Reihe 6, Schwarz Reihe 1

### Zugregeln

- Vorwärts 1 Feld (wenn frei)
- Diagonal schlagen möglich · Kein Doppelschritt

### Gewinnbedingungen

- Gegnerische Grundlinie erreichen
- Alle Figuren des Gegners schlagen
- Gegner kann nicht mehr ziehen

### Bewertungsfunktion

- Figurenzahl + Fortschritt + Zentrumskontrolle

## Bauernschach

6×6 · Weiß (Mensch) vs. Schwarz (KI)

---

Bauernschach ist eine vereinfachte Schachvariante, bei der nur Bauern auf einem 6×6-Feld gespielt werden.

### Aufbau

- Weiß (Mensch) startet in Reihe 6, Schwarz (KI) in Reihe 1. Beide Seiten haben je sechs Bauern.

### Zugregeln

- Ein Bauer kann **ein Feld vorwärts** ziehen, wenn das Zielfeld frei ist.
- **Diagonal schlagen**: Ein Bauer kann diagonal nach vorne ziehen, wenn dort ein gegnerischer Bauer steht.
- Kein Doppelschritt – anders als im echten Schach.

### Gewinnbedingungen

- Die gegnerische **Grundlinie erreichen** (Umwandlung).
- **Alle Figuren** des Gegners schlagen.
- Der Gegner kann **keinen Zug mehr** machen.
- **Kurz zeigen wie das Spiel aussieht (Screenshot auf der Folie).**

## Bewertungsfunktion Bauernschach

*Figurenzahl · Fortschritt · Zentrum*

---

- Figurenzahl: +10 pro eigener Bauer, -10 pro Gegner
- Fortschritt:  $+row \times 2 \rightarrow$  KI wird für Vorwärtsbewegung belohnt
- Zentrum: +0 bis +3 (Zentralposition strategisch wertvoller)
- Positiv = gut für KI (Schwarz) · Negativ = gut für Mensch
- CENTER\_BONUS-Matrix definiert Feldwertigkeit

## Bewertungsfunktion Bauernschach

Heuristik: *Figurenzahl · Fortschritt · Zentrum*

---

Die Bewertungsfunktion gibt der KI einen numerischen Score für jede Spielposition. Ein **positiver Wert** bedeutet, die KI steht gut; ein negativer, dass der Mensch im Vorteil ist.

■ **Figurenzahl (+10 pro eigener Bauer):** Jeder eigene Bauer zählt +10, jeder gegnerische -10. Wer mehr Figuren hat, steht besser.

■ **Fortschritt (+row×2):** Je weiter ein Bauer der KI nach vorne gerückt ist, desto mehr Punkte bekommt er. Das motiviert die KI, vorwärtszuspielen.

■ **Zentrumskontrolle (+0 bis +3):** Felder in der Mitte des Bretts sind strategisch wertvoller. Die CENTER\_BONUS-Matrix vergibt Boni von 0 (Rand) bis 3 (Zentrum).

■ **Kurz erklären warum eine gute Heuristik wichtiger ist als eine höhere Suchtiefe.**

## Tic-Tac-Toe (4 gewinnt)

6×6 · Mensch (X) vs. KI (O)

---

### Spielregeln

- Abwechselnd Stein setzen · Mensch (X) beginnt
- 4 in einer Reihe gewinnt (H/V/Diagonal)
- Unentschieden bei vollem Feld

### KI-Optimierungen

- Kandidatenzüge: nur Nachbarfelder (36 → 8–15 Züge)
- Center-Sorting: Zentrumsnähe für Alpha-Beta

### Bewertungsfunktion

- Dreier-Reihe: +50 / -50 · Zweier-Reihe: +10 / -10
- Mittelfeld-Bonus

## Tic-Tac-Toe (4 gewinnt)

6×6 · Mensch (X) vs. KI (O)

Die zweite Spielvariante: Vier in einer Reihe auf einem 6×6-Feld – eine Erweiterung des klassischen Tic-Tac-Toe.

### Spielregeln

- Mensch (X) und KI (O) setzen abwechselnd Steine, der Mensch beginnt.
- Wer zuerst **vier Steine in einer Reihe** hat – horizontal, vertikal oder diagonal – gewinnt.
- Bei vollem Feld ohne Gewinner: **Unentschieden**.

### KI-Optimierungen

- **Kandidatenzüge:** Statt alle 36 Felder zu prüfen, betrachtet die KI nur Nachbarmfelder besetzter Felder. Das reduziert die Züge auf 8–15.
- **Center-Sorting:** Kandidaten werden nach Zentrumsnähe sortiert. Alpha-Beta findet dadurch früher gute Äste und schneidet mehr ab.

### Bewertungsfunktion

- Dreier-Reihe: +50 / -50 · Zweier-Reihe: +10 / -10 · Mittelfeld-Bonus
- **Unentschieden kommt selten vor – erwähnen falls jemand fragt.**

## TTT Optimierungen

*Kandidatenzüge · Center-Sorting · shallow copy*

---

- Nur Nachbarmfelder besetzter Felder als Kandidaten
- Center-Sorting → Alpha-Beta findet früh gute Äste
- `[row[:]]` for row in board statt `deepcopy` → ~5× schneller

## TTT Optimierungen im Detail

*Kandidatenzüge · Center-Sorting · shallow copy*

---

### Kandidatenzüge

In einem 6×6-Feld gibt es anfangs 36 mögliche Züge. Die KI schränkt das auf Felder ein, die direkt neben einem bereits besetzten Feld liegen. Ein Stein ohne Nachbarn ist strategisch fast wertlos. Dadurch sinkt die Anzahl der zu prüfenden Züge auf typischerweise 8–15.

### Center-Sorting

Kandidaten werden vor der Auswertung nach Entfernung zum Mittelpunkt sortiert. Da Alpha-Beta bessere Züge zuerst sehen und mehr Äste abschneiden kann, beschleunigt das die Suche erheblich.

### shallow copy statt deepcopy

Das Spielfeld wird als [row[:] for row in board] kopiert – etwa 5× schneller als deepcopy(board), weil nur die Zeilenlisten flach kopiert werden. Für ein 6×6-Feld mit ganzen Zahlen ist das vollständig korrekt.

■ **Wer fragt: deepcopy wäre korrekt für verschachtelte Objekte, hier aber nicht nötig.**



## GUI & Threading

*daemon Thread · after(0, ...) · app\_state*

---

- `app_state{}` als globaler Zustand (prozedural, kein OOP)
  - `board`, `ai_thinking`, `difficulty`, `current_user`, ...
- KI-Zug in separatem Thread → GUI bleibt reaktionsfähig
- `daemon=True` → Thread endet mit Fenster
- `after(0, fn)` → UI-Updates sicher in tkinter-Hauptthread
- tkinter ist nicht thread-safe → NIE Widgets aus Thread ändern

## GUI & Threading

*daemon Thread · after(0, ...) · app\_state*

---

### app\_state – globaler Zustand

Statt Klassen wird der gesamte Programmzustand in einem einzigen Dictionary `app_state` gespeichert. Das enthält das aktuelle Spielfeld, den eingeloggten Nutzer, die gewählte Schwierigkeit, und ob die KI gerade denkt.

### KI-Zug in separatem Thread

Der MiniMax-Algorithmus kann bei höheren Schwierigkeitsgraden mehrere Sekunden rechnen. Würde er im Hauptthread laufen, würde das GUI einfrieren. Deshalb läuft der KI-Zug in einem separaten Thread.

- **daemon=True:** Der Thread wird automatisch beendet wenn das Fenster geschlossen wird – kein Aufräumen nötig.

### after(0, callback) – Thread-Safety

tkinter ist nicht thread-safe. Widgets dürfen nur aus dem Hauptthread verändert werden. Deshalb ruft der KI-Thread am Ende `after(0, fn)` auf – das marshallt den GUI-Update sicher in den tkinter-Hauptthread zurück.

- **Gute technische Frage vorhersagen: 'Warum kann man tkinter nicht aus dem Thread aufrufen?' → Event-Loop-Architektur.**

## Unittests

*TC-01 bis TC-15 · test\_all.py · pytest-kompatibel*

---

- TC-01–03: Passwort-Hashing (Generierung, Verifikation, Salt)
- TC-04–06: Benutzer-DB (Registrierung, Duplikat, Abfrage)
- TC-07–08: Spielergebnisse (Speichern, Bestenliste)
- TC-09–10: Bauernschach (Züge, Anwenden, Gewinner)
- TC-11–12: Tic-Tac-Toe (4-in-Reihe, Kandidatenzüge)
- TC-13–15: MiniMax (offens. Zug, Gewinn erkennen, Blockieren)

## Unittests

*TC-01 bis TC-15 · test\_all.py · pytest-kompatibel*

---

Das Projekt umfasst 15 Unittests, die alle Kernfunktionen abdecken:

- **TC-01–03 (Passwort-Hashing):** Prüft ob ein Hash korrekt generiert wird, ob die Verifikation funktioniert, und ob zwei gleiche Passwörter verschiedene Hashes erzeugen.
  - **TC-04–06 (Benutzer-DB):** Registrierung eines neuen Nutzers, Erkennung von Duplikaten, Abfrage des Nutzerprofils.
  - **TC-07–08 (Spielergewinne):** Speichern eines Ergebnisses, korrekte Bestenliste abrufen.
  - **TC-09–10 (Bauernschach):** Gültige Züge generieren, Zug anwenden, Gewinner erkennen.
  - **TC-11–12 (Tic-Tac-Toe):** Vier-in-Reihe korrekt erkennen, Kandidatenzüge korrekt einschränken.
  - **TC-13–15 (MiniMax-KI):** Offensichtlichen Gewinnzug finden, drohenden Gewinn des Gegners blockieren.
- Erwähnen dass die Tests mit `'python -m pytest test_all.py'` ausgeführt werden.

## Test-Infrastruktur

*unittest · Temp-DB-Isolation · Mocking*

---

- DBTestCase: jeder Test bekommt frische Temp-DB
- tempfile.mkstemp('.db') → isolierte Datenbankdatei
- patch.object(database, 'DB\_PATH') → keine Produktions-DB
- assertEquals testet 4-in-Reihe-Erkennung direkt

## Test-Infrastruktur

*unittest · Temp-DB · Mocking*

---

Damit Tests unabhängig voneinander und von der echten Datenbank laufen, verwenden wir Isolation durch temporäre Datenbanken:

### DBTestCase

Jeder Testfall erbt von DBTestCase. In setUp() wird per tempfile.mkstemp() eine neue temporäre DB-Datei erstellt. Nach dem Test wird sie wieder gelöscht. So beeinflussen sich Tests nicht gegenseitig.

### patch.object

Mit unittest.mock.patch.object wird database.DB\_PATH auf den temporären Pfad umgeleitet. Der getestete Code läuft unverändert, schreibt aber in die Temp-DB statt in die echte games.db.

### pytest-Kompatibilität

Die Tests sind als unittest.TestCase geschrieben, können aber direkt mit pytest ausgeführt werden – pytest erkennt unittest-Tests automatisch.

■ **Falls jemand fragt: Mocking verhindert dass Tests die Produkt-DB verändern oder voneinander abhängen.**

## Zusammenfassung

---

- ■ Zwei vollständige Strategiespiele auf 6×6-Feld
- ■ Wiederverwendbarer MiniMax + Alpha-Beta-Pruning
- ■ Sichere Authentifizierung: SHA-256 + Salt
- ■ SQLite: keine externen Abhängigkeiten
- ■■ tkinter-GUI: Windows / macOS / Linux
- ■ 15 Unittests sichern Kernfunktionen ab

## Zusammenfassung

*Was haben wir erreicht?*

---

Zusammenfassend haben wir alle wesentlichen Projektziele erreicht:

- ■ **Zwei vollständige Strategiespiele** auf einem 6×6-Spielfeld – vollständig spielbar gegen eine KI.
  - ■ **Wiederverwendbarer MiniMax-Algorithmus** mit Alpha-Beta-Pruning – eine Engine für alle Spiele.
  - ■ **Sichere Authentifizierung** mit SHA-256-Hashing und zufälligem Salt – keine Klartextpasswörter.
  - ■ **SQLite-Datenbank** ohne externe Abhängigkeiten – die DB wird automatisch erstellt.
  - ■■ **tkinter-GUI** mit eigenem Design-System – läuft auf Windows, macOS und Linux.
  - ■ **15 Unittests** sichern die Kernfunktionen ab und können jederzeit ausgeführt werden.
- **Ruhig sprechen – das ist die Zusammenfassung, kein neues Material.**



## SOLL/IST Feature-Vergleich

*Lastenheft → Pflichtenheft → Umsetzung → Test*

---

### MUSS-Ziele: 12/12 ■

- 2 Spiele · 6×6 Brett · 5 Schwierigkeitsgrade
- Login · Registrierung · Gastmodus · Spielabbruch · Regelanzeige
- Bestenliste · SHA-256-Hashing · MiniMax-KI · Unittests

### SOLL-Ziele: 2/2 ■

- Alpha-Beta-Pruning · SQL-Bestenliste

### KANN-Ziele: 2/4 ■

- ■ Sprachwechsel EN/DE · Sprachpräferenz in DB
- ■ Dame (drittes Spiel) · Alt-KI-Schnittstelle (zeitbedingt)

## SOLL/IST Feature-Vergleich

*Lastenheft → Pflichtenheft → Umsetzung → Test*

---

### MUSS-Ziele: 12 von 12 ■

Alle Pflichtenanforderungen wurden vollständig umgesetzt: beide Spiele, 5 Schwierigkeitsgrade, Login, Registrierung, Gastmodus, Spielabbruch mit Dialog, Regelanzeige, Bestenliste, SHA-256-Hashing, MiniMax-KI und Unittests.

### SOLL-Ziele: 2 von 2 ■

Beide Soll-Anforderungen wurden erfüllt: Alpha-Beta-Pruning als MiniMax-Optimierung und die Bestenliste über SQL-Abfragen aus der Datenbank.

### KANN-Ziele: 2 von 4 ■

- ■ **Sprachwechsel EN/DE** und Sprachpräferenz in der Datenbank sind implementiert.

- ■ **Dame als drittes Spiel** und eine alternative KI-Schnittstelle wurden aus Zeitgründen nicht umgesetzt.

Insgesamt: Alle MUSS- und SOLL-Anforderungen sind erfüllt. Die nicht umgesetzten KANN-Ziele waren von Anfang an als optional eingestuft.

■ **Nicht entschuldigen für die KANN-Ziele – sie waren optional und das ist legitim.**

## SOLL/IST Projektplanung

*Wasserfallmodell · 2 Meetings · Discord & WhatsApp*

---

- Analysephase (Woche 1: 20.04.–24.04.) ■ plangemäß
- Implementierung (27.04.–04.05.) ■ +1 Tag (01.05. Feiertag)
- Test & Doku: parallel zur Impl. statt sequenziell ■
- Abschluss (Woche 3) ■ plangemäß
- Discord-Channel statt Kanban-Board
- 2 Team-Meetings: 27.04. & 29.04. je 10:00 Uhr

## SOLL/IST Projektplanung

*Wasserfallmodell · 2 Meetings · Discord*

---

Das Projekt wurde nach dem Wasserfallmodell geplant und in drei Phasen durchgeführt:

- **Analysephase (Woche 1: 20.04.–24.04.)** ■ plangemäß: Pflichtenheft, Work Breakdown Structure und Zeitplan wurden erstellt.
- **Implementierung (27.04.–04.05.)** ■ +1 Tag: Durch den Feiertag am 01. Mai wurde der 04. Mai als zusätzlicher Arbeitstag genutzt.
- **Test & Dokumentation** ■ parallel: Entgegen der ursprünglichen Planung liefen Test und Dokumentation parallel zur Implementierung – das war pragmatischer.
- **Abschluss (Woche 3)** ■ plangemäß: Abschlusspräsentation wie geplant.

### Abweichungen

Statt eines Kanban-Boards wurde ein Discord-Channel als leichtgewichtige Alternative genutzt. Die zwei geplanten Team-Meetings fanden am 27.04. und 29.04. jeweils um 10:00 Uhr statt.

■ **Abweichungen positiv formulieren: 'wir haben uns angepasst' statt 'wir haben den Plan nicht eingehalten'.**

## Benutzeroberfläche (GUI)

*tkinter · HHBKTendo Design System · Dark Theme*

---

### Screen-Flow

- ① Login → Login / Register / Als Gast spielen
- ② Hauptmenü → Spiel · Schwierigkeit · Play · Bestenliste
- ③ Spielscreen → 6×6 Brett · Züge per Klick

### Design-System (Farben)

- #0D0D1A Deep Navy (BG) · #E94560 Hot Pink (Primär)
- #6C3FC5 Royal Purple · #00D4FF Cyber Cyan (Akzent)

### tkinter-Besonderheiten

- Buttons als tk.Label + Bindings (macOS-Bug: bg/fg)
- KI in separatem Thread · after(0, cb) für GUI-Rückkehr

# Benutzeroberfläche (GUI)

tkinter · HHBKTendo Design System · Dark Theme

---

## Screen-Flow

- ① **Login-Screen:** Nutzer können sich einloggen, registrieren oder als Gast spielen.
- ② **Hauptmenü:** Spiel auswählen, Schwierigkeitsgrad einstellen, starten oder Bestenliste öffnen.
- ③ **Spielscreen:** 6×6-Spielfeld, Züge per Mausclick, KI-Zug wird nach kurzer Denkzeit angezeigt.

## Design-System

- Deep Navy (#0D0D1A) als Hintergrund · Hot Pink (#E94560) für Primärelemente
- Royal Purple (#6C3FC5) als Sekundärfarbe · Cyber Cyan (#00D4FF) für Hover und Akzente

## Technische Besonderheiten

- Buttons sind tk.Label mit Bindings implementiert – weil tk.Button auf macOS bg/fg-Farben ignoriert.
- KI-Zug läuft in separatem Thread, GUI bleibt reaktionsfähig. after(0, callback) bringt das Ergebnis sicher in den Hauptthread zurück.
- Falls Demo: kurz das Dark Theme und die Farbgebung erwähnen.

## Quellen & Werkzeuge

---

### Entwicklungsumgebungen

- PyCharm (JetBrains) · Visual Studio Code (Microsoft)

### KI-Assistenten

- Claude (Anthropic) · ChatGPT (OpenAI)

### Python-Bibliotheken (alle stdlib außer Pillow)

- tkinter · sqlite3 · hashlib · threading · Pillow (GIF)

### Quellen

- Wikipedia: MiniMax · Alpha-Beta-Pruning
- HHBK Lernfeld 5 – Unterrichtsmaterial
- docs.python.org – Python-Dokumentation

## Quellen & Werkzeuge

---

### Entwicklungsumgebungen

- PyCharm (JetBrains) und Visual Studio Code (Microsoft)

### KI-Assistenten

- Claude (Anthropic) und ChatGPT (OpenAI) wurden zur Unterstützung bei der Entwicklung genutzt.

### Python-Bibliotheken

- Alle Bibliotheken außer Pillow sind Teil der Python-Standardbibliothek: tkinter, sqlite3, hashlib, threading, unittest, tempfile.
- Pillow wurde für GIF-Animationen in der GUI verwendet.

### Quellen

- Wikipedia: Artikel zu MiniMax-Algorithmus und Alpha-Beta-Pruning
- HHBK Lernfeld 5 – Unterrichtsmaterial
- docs.python.org – offizielle Python-Dokumentation

### Projektkoordination

- Zwei Team-Meetings (27.04. & 29.04., 10:00 Uhr), tägliche Kommunikation über Discord und WhatsApp.



## Fazit – Projektverlauf

---

### Team & Rollen

- Lucas – Projektleitung, auth.py, Testing
- Jesse – database.py, Pitch, Testing
- Arda – database.py, Testing
- Uilliam – GUI (main.py), Design
- Jakub – pawn\_chess.py · Niklas – minimax.py

### Highlights

- Spielunabhängige MiniMax-Schnittstelle
- Reibungslose Teamarbeit dank klarem Pflichtenheft

### Lessons Learnt

- Kanban-Board früher einsetzen
- Schnittstellen detaillierter im Pflichtenheft definieren

### Ausblick

- Dame als drittes Spiel · Sprach-Auswahl ausbauen

## Fazit – Projektverlauf

*Team · Highlights · Lessons Learnt*

---

### Team & Rollenverteilung

- Lucas – Projektleitung, auth.py, Testing
- Jesse – database.py, Pitch, Testing · Arda – database.py, Testing
- Uilliam – GUI (main.py), Design · Jakub – pawn\_chess.py
- Niklas – minimax.py · Jamie – tictactoe.py

### Highlights

- Die spielunabhängige MiniMax-Schnittstelle war die eleganteste technische Lösung – ein KI-Code für alle Spiele.
- Die Teamarbeit verlief reibungslos dank eines klaren Pflichtenhefts und gegenseitiger Unterstützung.

### Lessons Learnt

- Ein Kanban-Board früher einsetzen hätte die Aufgabenverteilung transparenter gemacht.
- Schnittstellen zwischen Modulen detaillierter im Pflichtenheft definieren.

### Ausblick

- Dame als drittes Spiel wäre mit dem bestehenden Callback-System einfach umsetzbar.
  - Sprach-Auswahl im Menü und weiterer Ausbau des Design-Systems.
- **Zum Abschluss: ruhig und selbstbewusst schließen. Das Projekt ist vollständig.**